AI Engineer Take-Home Exercise: Gen AI Agent for Contextual CVE Analysis

Context:

Rad Security is leveraging the power of Generative AI and large language models (LLMs) to revolutionize cybersecurity workflows. Analyzing vulnerabilities (CVEs) during an incident requires understanding complex relationships between affected systems, attacker behavior, and vulnerability details. The volume and complexity of CVE data, combined with noisy and incomplete incident information, make manual analysis inefficient. We are leveraging AI to automate and enhance this process.

Problem:

Design and implement a *working* AI agent or agent workflow that can analyze incident context and relevant CVE data to identify and prioritize the most impactful vulnerabilities. The agent should leverage an LLM for reasoning and potentially interact with simulated tools or data sources to gather information and formulate its analysis, going beyond simple database lookups.

Objective:

Your task is to design and implement a functional Gen AI agent component using a framework like LangChain, LangGraph, LlamaIndex, or a similar approach. The agent should take simulated incident data (like the provided sample) and access relevant CVE information (which you can simulate) to perform the following:

1. **Understand Incident Context:** Reason about the affected assets, observed TTPs, and initial findings.
2. **Identify Relevant CVEs:** Determine which CVEs are potentially relevant based on the incident context and affected software/hardware, using LLM reasoning and potentially querying data sources.
3. **Prioritize CVEs:** Assess the risk and impact of relevant CVEs *in the context of the specific incident*, going beyond standard scores like CVSS.
4. **Generate Analysis:** Provide a brief, human-readable explanation of *why* certain CVEs are prioritized, linking them back to the incident details.

Requirements:

1. **Agent Design & LLM Application:** Describe the architecture of your Gen AI agent. Explain how the LLM is used for reasoning, understanding context, and

generating output. Detail the agent's workflow or state machine (if using LangGraph or similar).

2. **Tool Use (Simulated/ Open source):** Identify potential tools or data sources the agent would need to interact with (e.g., a simulated/ open CVE database lookup tool, a simulated/open asset inventory query tool, a simulated/open threat intelligence lookup tool). Explain how the agent would decide which tool to use and how it would interpret the results. Your implementation should *simulate* interaction with at least one such tool/data source.
3. **Implementation (Core Agent Logic):** Provide the source code for your working Gen AI agent component using your chosen framework (LangChain, LangGraph, LlamaIndex, etc.). The code should demonstrate the agent's core reasoning loop, interaction with simulated tools/data, and processing of the sample incident data to produce a prioritized output. You can use a local or publicly available LLM API (clearly state which one and any associated costs/keys needed, or use a mock LLM if preferred). Python is preferred.
4. **Data Preparation for LLM:** Explain how you prepare the incident data and relevant CVE information to be provided as context to the LLM. How do you handle the context window limitations of the LLM?
5. **Output & Explainability:** Describe the output of your agent, focusing on the prioritized list and the generated analysis/justification. How do you design the agent's prompts or workflow to ensure the LLM provides clear, traceable reasoning for its conclusions?
6. **Evaluation Metrics (Gen AI Focus):** How would you evaluate the effectiveness and quality of the *agent's output and reasoning*? What specific metrics (e.g., related to relevance of generated insights, correctness of tool use, coherence of explanation) are most relevant? How would you collect ground truth or evaluate subjective quality?
7. **Production Considerations:** Discuss key challenges and considerations for deploying and maintaining this type of Gen AI agent in a production security environment (e.g., LLM cost and latency, prompt engineering robustness, tool reliability, safety/bias, monitoring agent performance).

Submission:

Please submit:

1. A document (PDF or Markdown) outlining your design, addressing the specific questions below, and explaining your implementation choices. Include diagrams if they help explain your agent's architecture and workflow.

2. The source code for your working Gen AI agent component. Include clear instructions on how to set up and run the code, including any dependencies and how to configure the LLM (API key, local model path, or mock). You should also provide sample input data (using the provided incident data and simulated CVE data/tools) that your component can process.

Focus on demonstrating your expertise in designing and implementing solutions using modern Gen AI agentic patterns, leveraging LLMs and relevant frameworks, while applying them effectively to the cybersecurity domain.

Sample Incident Alert Data:

Please refer to the separate document titled "Synthetic Incident Alert Data (25 Points)" for the dataset to be used as input for your working agent. This dataset contains 25 simulated incident scenarios.
https://docs.google.com/document/d/1GbN1VoBt7h7qqgAokhhXXd1tgOpqeICJKoSSttN3tWE/edit?usp=sharing

Specific Questions for Candidate Responses (for comparison):

To help us compare candidates' approaches, please specifically address the following questions within your design document:

1. Describe your agent's architecture and workflow. How does the LLM orchestrate the process of analyzing incident data and identifying relevant/prioritizing CVEs?
2. Detail the prompts or prompting strategy you use to guide the LLM's reasoning for CVE relevance and prioritization based on incident context.
3. Explain how your agent uses or simulates interacting with external tools or data sources (e.g., CVE database lookup). How does the agent decide *when* to use a tool and *how* it incorporates the tool's output?
4. How do you prepare and provide the incident data and relevant CVE information to the LLM within its context window?
5. What is the final output of your agent, and how is the LLM's reasoning process (chain of thought) presented to the analyst for explainability and trust?
6. What specific metrics would you use to evaluate the performance and quality of your *Gen AI agent's output and reasoning* in this task? How would you collect data for this evaluation?
7. Discuss the main challenges you anticipate in making this agent robust and

reliable in a production security environment, particularly related to LLM behavior and tool interaction.